

Learning to Defer with Semantic Data Drift Analysis

A Simple Guide to the Idea, the Approach, and the Build Plan

Semester V Machine Learning Mini-Project

1. What Is This Project, In Simple Words

Telecom companies (and similar subscription businesses) use machine learning models to guess which customers might cancel their service (“churn”), so they can try to retain them. The problem: these models are trained once, on old data, and the world keeps changing underneath them — prices change, competitors launch new offers, customer habits shift. The model doesn't know this is happening. It just keeps giving confident answers, even after those answers have quietly become unreliable.

This project builds a system that sits on top of a normal churn-prediction model and does one specific new thing: it watches for the moment the model's reasoning stops making sense, flags exactly which customer cases are affected and why, and hands only those specific cases over to a human analyst — instead of either trusting the model blindly or making a human double-check everything.

In one sentence: we don't just predict churn — we teach the system to notice when its own predictions can no longer be trusted, and to ask for human help only on those specific cases.

1.1 Who This Actually Helps

- Retention teams / call-center analysts — today they get a list of “risky” customers with no way to know if that risk score is still trustworthy. Our system tells them which specific customers need a second look, and why.
- The team maintaining the model — today they only find out the model has gone stale through a scheduled check-up or a customer complaint. Our system raises a flag automatically, earlier, with a specific reason attached.

1.2 An Important Honest Limit (the “ice cream shop” point)

Imagine an ice cream shop loses a customer simply because it ran out of one specific flavor that day. No model trained on general customer data (how long they've been a customer, what they pay, etc.) could ever predict that — it's a one-off, situational reason, not a pattern. Our project does not claim to solve this kind of problem. It only works for businesses where customers have an ongoing account and a clear “cancel/renew” moment — telecom, internet providers, insurance, subscription apps. This is a deliberate boundary, not an oversight, and it's important to say this out loud rather than pretend the system does more than it does.

2. How We Are Approaching It

2.1 The Four Things That Make This More Than “Just Combining Known Techniques”

Individually, SHAP (explainability), drift detection, uncertainty scoring, and human hand-off ('learning to defer') are all existing, well-known ideas. Simply gluing them together isn't innovation by itself. Here is what is actually new about how we use them:

A) Explanation drift as an instant, per-customer trigger — not just a warning dashboard

Most existing systems that watch for “drift” just tell an engineer “something changed, go look into it” — like a smoke detector that beeps and waits for a person to react. Our system instead checks every single prediction and, the moment it looks unreliable, immediately routes that one case to a human — no one has to notice an alert first.

B) A combined risk score that treats “uncertain AND drifting” as extra dangerous

A model being unsure is one warning sign. The data changing underneath it is a second, separate warning sign. Our idea is that when both happen on the same customer at the same time, that's riskier than either alone — similar to how being tired and driving on a wet road is more dangerous than the two risks simply added up. This is a hypothesis we test properly with real experiments, not something we assume is true without proof.

C) Telling the human WHY a case was flagged, not just handing it over blindly

Most systems that hand a case to a human just say “I'm not sure, you decide” with zero extra information. Ours attaches the specific customer details that behaved unusually, so the analyst knows exactly where to look first instead of starting from scratch.

D) Deciding when to retrain the model based on real evidence, not a fixed calendar

Most companies retrain their models on a routine schedule (e.g., every 3 months) whether or not anything actually changed. Our system instead watches how often the flagged cases turn out to be genuinely wrong, and raises a “time to retrain” signal only when there's real evidence the model has gone stale — similar to a car that tells you to change the oil based on its actual condition, not a fixed date.

2.2 The Domain We Are Focusing On

We are deliberately scoping this project to B2C subscription/contract-based churn — telecom being our specific example, with the same idea applying to similarly structured businesses (internet providers, insurance, subscription apps). We are explicitly not claiming this works for retail/walk-in businesses (like the ice cream shop example), where there's no clean “cancel” event and causes are often one-off situations rather than patterns.

2.3 Technology Stack

Layer	What We Use	In Simple Terms
Programming language	Python	The language everything is built in
Main prediction model	XGBoost	A well-tested algorithm that's good at learning patterns from spreadsheet-style data
Making the model's confidence trustworthy	Platt scaling	Converts the model's raw score into a real, reliable probability
Explaining predictions	SHAP	Shows which customer details influenced a specific prediction the most
Detecting when things have changed	Mann-Whitney U statistical test	A way to check if two groups of data are meaningfully different, used to catch drift
Extra real-world context	FRED India CPI data	Adds inflation/economic context from India to the customer data
Dashboards for humans	Power BI	Turns our results into visual charts analysts and the ML team can actually use

3. Phase-by-Phase Implementation Plan

Each phase below builds directly on the one before it — don't skip ahead, since every later phase assumes the previous one's output is ready and correct.

Phase 1 — Get and Clean the Data

What happens: download the Telco customer dataset, fix small data problems (like a billing column that's stored as text instead of numbers), and turn all the “Yes/No” and category columns into numbers a model can understand.

Why it matters: a model can only be as good as the data you feed it — skipping this step causes confusing errors much later, when they're harder to trace back.

- Load the raw customer data and inspect it for missing or broken values.
- Fix the known issue where total-charges is stored as text for brand-new customers.
- Convert all category columns (like contract type) into a number format the model can use.

Phase 2 — Invent a Rough Timeline and Add Economic Data

What happens: since this dataset has no real signup date, we calculate an approximate one from how long each customer has stayed with the company (tenure). We then attach India's inflation data (CPI) to each customer, based roughly on when they joined.

Why it matters: our whole project is about detecting change over time — without some sense of time, there's nothing to compare 'before' and 'after' on. We are upfront that this timeline is a reasonable estimate, not a real historical record.

- Calculate an approximate signup date for each customer from their tenure.
- Sort customers into 10 ordered groups (“windows”), oldest to newest.
- Fetch India CPI data from the FRED database and attach it to each customer's approximate signup month.

Phase 3 — Train the First (Baseline) Prediction Model

What happens: train a standard XGBoost model on the earliest window of data, and check how well it predicts churn using normal accuracy measures.

Why it matters: this becomes our reference point — everything we add later (drift detection, deferral) is measured against how much better it does compared to this plain, ordinary model.

- Train an XGBoost model using only the first time window.
- Measure accuracy, precision, recall, and a combined score called F1.
- Apply Platt scaling so the model's confidence numbers are trustworthy, not just a rough ranking.

Phase 4 — Add Explainability (SHAP)

What happens: for every prediction, calculate which customer details (e.g. contract type, monthly charges) influenced that specific prediction the most.

Why it matters: this is the foundation the whole drift-detection idea is built on — without knowing which features mattered, we can't later detect when their importance starts changing.

- Run SHAP on the trained model to get feature-importance values for every customer.
- Save these values — they become the baseline “normal behavior” we compare future windows against.

Phase 5 — Build the Drift Detector

What happens: compare each new time window's SHAP values against the very first (baseline) window's SHAP values, using a statistical test, to catch when a feature's importance has changed in a way that's unlikely to be random chance.

Why it matters: this is literally the “noticing something changed” part of the whole project — without it, the system has no way to know it should distrust itself.

- Run the Mann-Whitney U statistical test comparing SHAP values across windows, per feature.
- Test this on artificially created (synthetic) drift first, where we know exactly what changed and when, to confirm the detector actually works before trusting it on real data.

Phase 6 — Build the Deferral (Hand-off) Logic

What happens: combine the model's confidence score with the drift score into one overall “risk” number per customer, and decide which customers get sent to a human analyst, based on how many cases the analyst team can realistically review.

Why it matters: this is the actual decision-making core of the project — it's what turns “we detected a problem” into “we did something useful about it.”

- Build the combined score using both the confidence and the drift measurement.
- Decide a review “budget” (e.g., only the riskiest 10% of cases go to a human, since analysts have limited time).
- Attach the specific SHAP reasons to every case that gets sent to a human.

Phase 7 — Test Whether Our Idea Actually Works (Evaluation)

What happens: compare our combined scoring method against simpler alternatives (using only confidence, only drift, or a basic either/or rule) to see if it genuinely catches more real mistakes.

Why it matters: this is the proof step. Without this, our “innovation” is just an untested opinion. This step turns it into an evidence-backed claim, one way or the other.

- Run all scoring methods side by side on the same data.
- Measure how many of the flagged cases were actually going to be wrong predictions (this is the main number that matters).
- Be honest in the report about the result, whether it favors our method strongly, partially, or not at all.

Phase 8 — Build the Power BI Dashboards

What happens: turn our results into visual charts — one dashboard for the analyst team (their daily list of flagged customers, with reasons) and one for the ML team (tracking whether the model is drifting and when it might need retraining).

Why it matters: a working system nobody can see or use isn't very useful in practice — this step makes the project's output usable by an actual human, not just a script that prints numbers.

- Build the analyst-facing dashboard: flagged customers, why they were flagged, and outcomes over time.
- Build the ML-team-facing dashboard: which features are drifting, and a simple status light showing whether it's time to retrain.

Phase 9 — Write Up and Present

What happens: pull together the results, the evidence from Phase 7, and the dashboards into the final report and presentation.

- Summarize what worked, what didn't, and be clear about the project's honest boundaries.
- Use the comparison results from Phase 7 as the central evidence for the innovation claim, not just a description of the architecture.

End of Document